

Agentic AI-Powered Database Migration Platform

Any-to-any migration across Oracle, MySQL and PostgreSQL using LangGraph, AWS Bedrock and pluggable tool adapters

Capstone Project | Rupak Banerjee

1. Problem statement

Dialect gap

Oracle, MySQL and PostgreSQL differ in data types, functions, PL/SQL vs PL/pgSQL, triggers and views.

Manual and slow

DBAs hand-rewrite thousands of objects. Error-prone, hard to repeat, hard to audit.

Silent data errors

Row loss or drift after a bulk load goes unnoticed without reconciliation.

Risky cutover

Switching the application has no approval gates and no safe rollback.

Need: an automated, auditable, human-supervised migration between any pair of Oracle, MySQL and PostgreSQL

2. Business use case



1

Lower licence cost

e.g. Oracle to PostgreSQL

2

Cloud modernization

Managed RDS on EKS, via
Terraform

3

Faster, repeatable

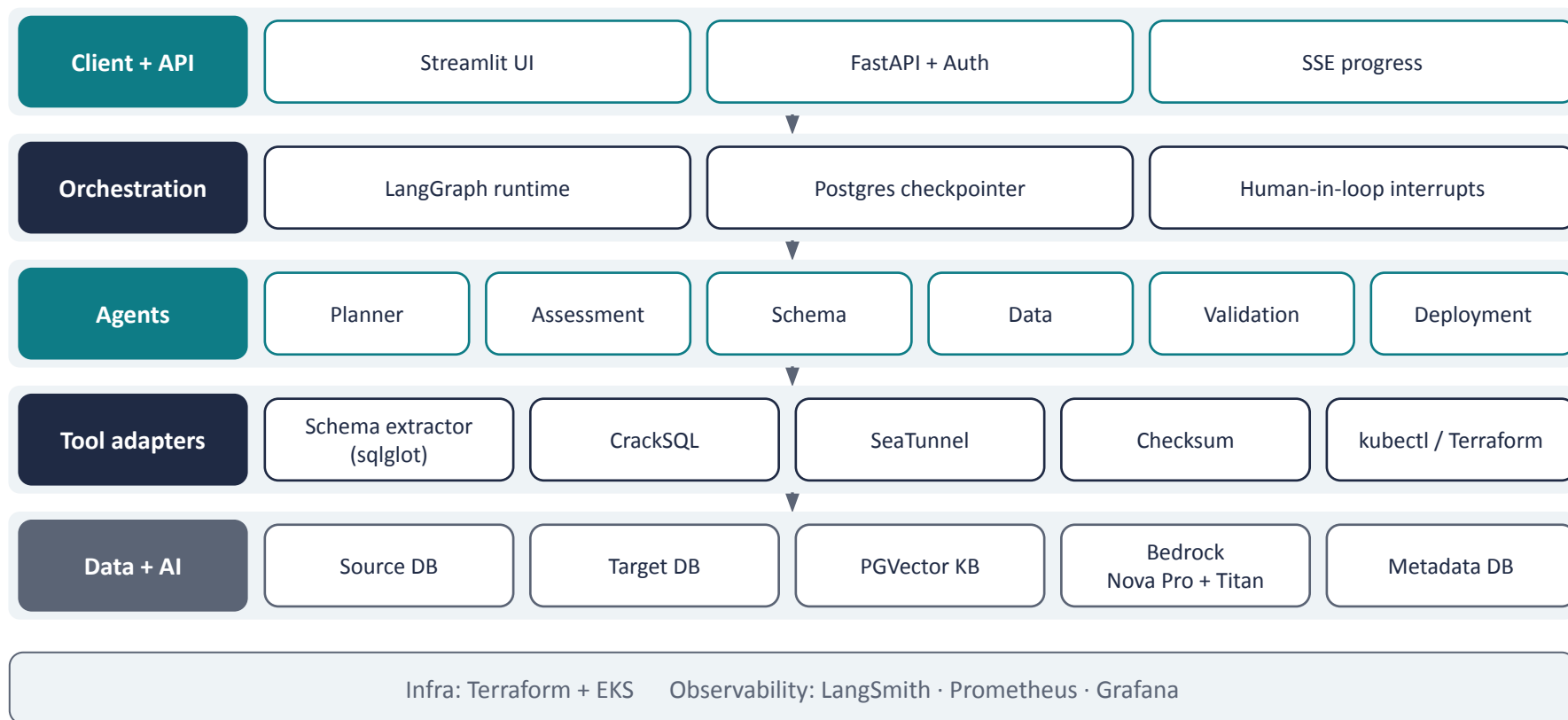
Agents replace manual
rewriting

4

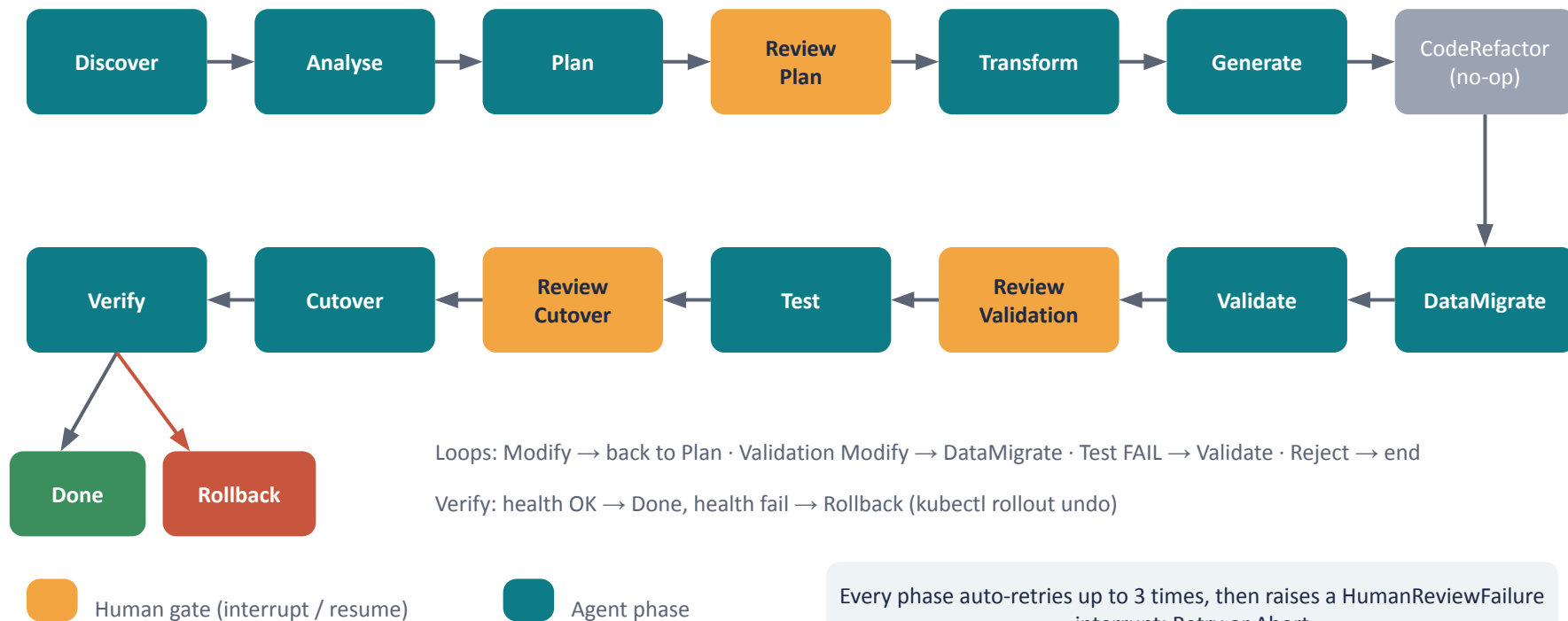
Audit-ready

Approvals, traces, rollback

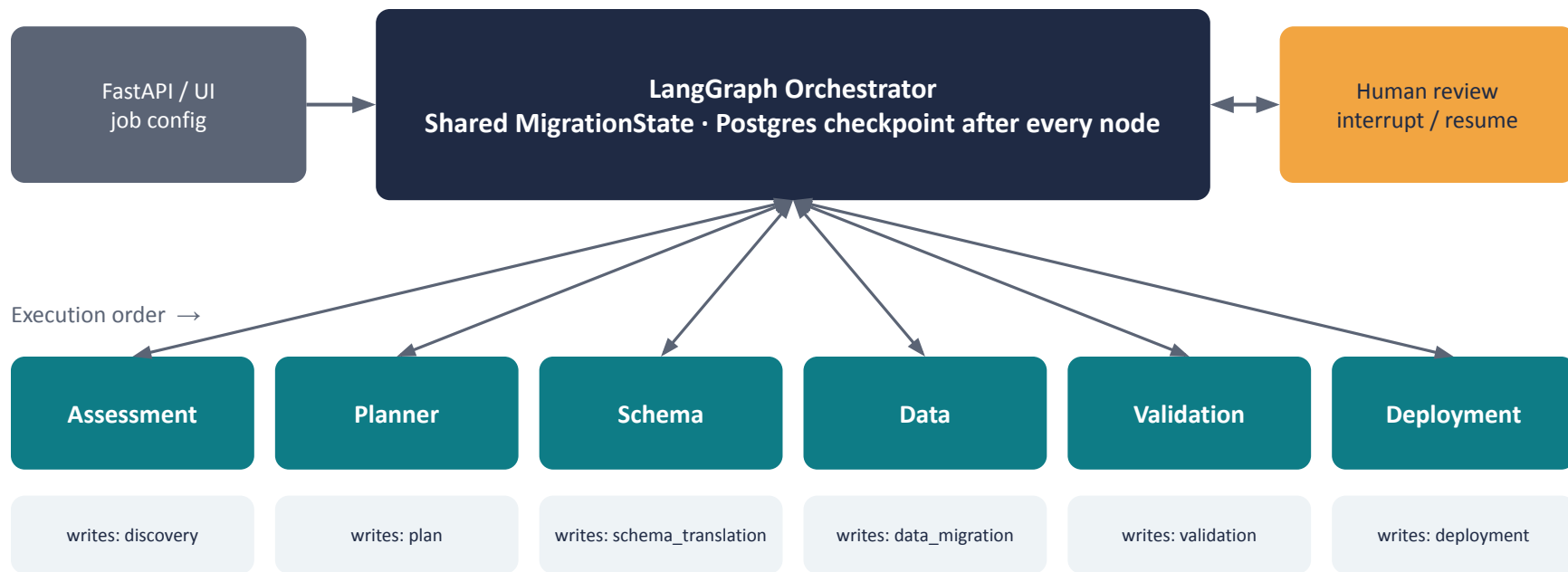
3. Architecture diagram



4. LangGraph workflow



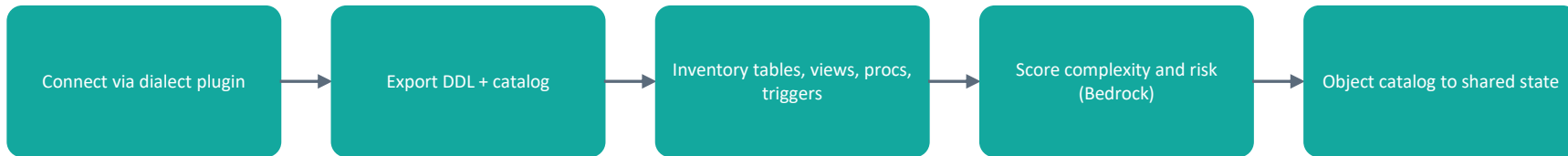
6. How agents communicate



Agents never call each other. Each one reads upstream fields from the state, does its work through its adapter, and writes its result back. A router function reads `state.status` to choose the next node.

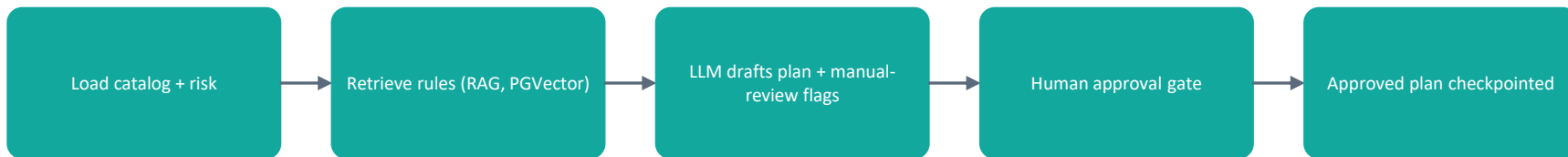
Agent flowcharts (1/3): Assessment and Planner

Assessment Agent



Reads source and target databases and builds a factual inventory with compatibility risks, so every later step starts from real schema data.

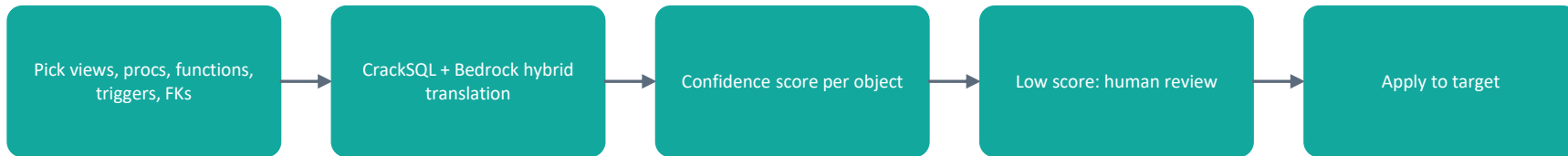
Planner Agent



Turns the inventory into an ordered migration plan using stored migration knowledge. Nothing proceeds until a person approves; state is checkpointed so jobs can resume.

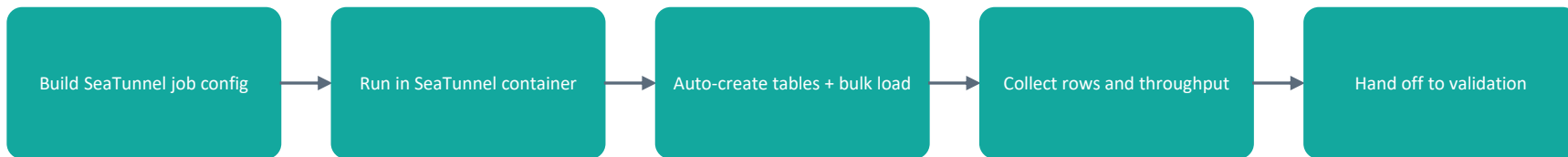
Agent flowcharts (2/3): Schema and Data

Schema Agent



Translates business logic between dialects. Table DDL is deliberately not translated here; SeaTunnel auto-creates tables. LLM output never auto-applies without a gate or validation.

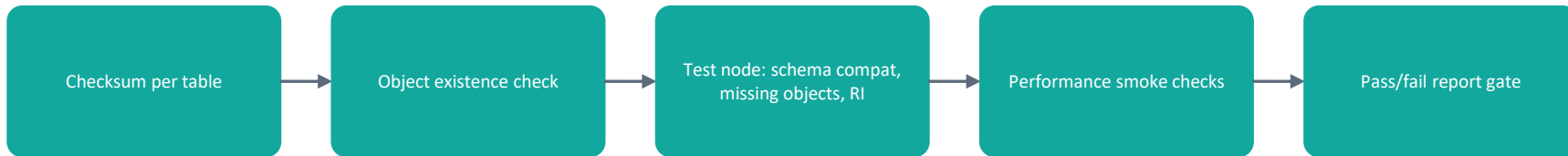
Data Agent



Moves the data in bulk through Apache SeaTunnel using JDBC drivers. Jobs run inside the container, so databases are reached via `host.docker.internal`.

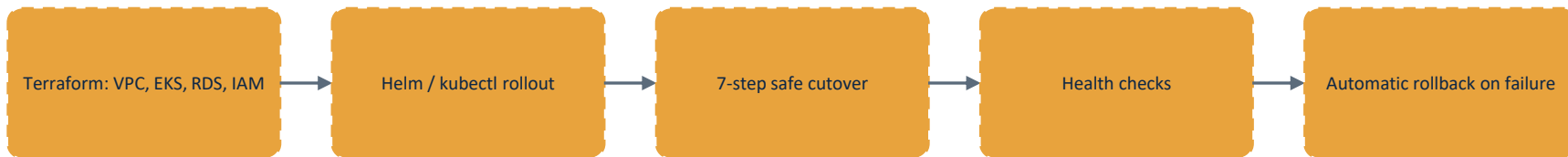
Agent flowcharts (3/3): Validation + Test and Deployment

Validation + Test Agent



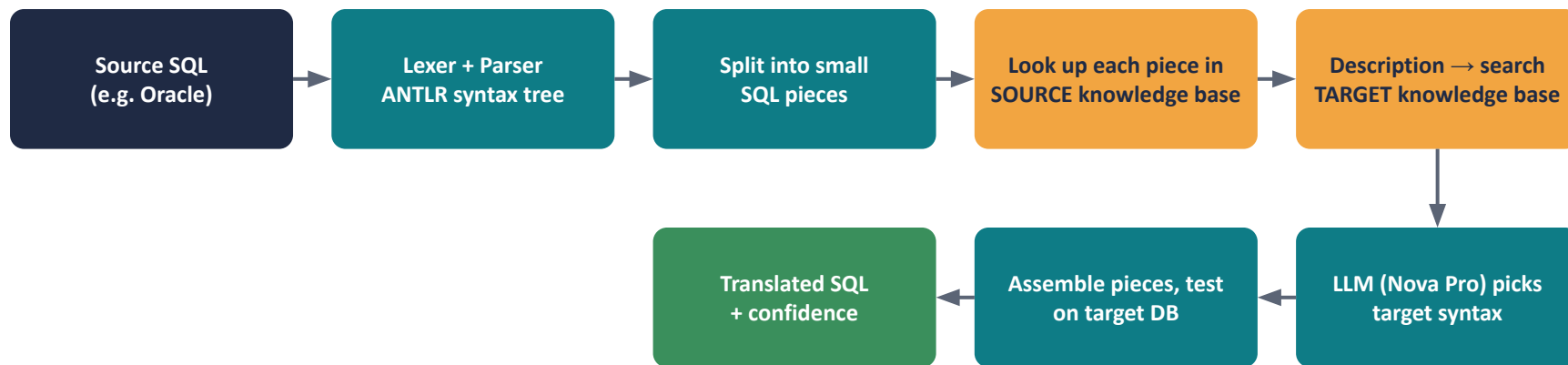
Proves the target matches the source before cutover. Results show in the Streamlit Validation and Test Report tabs and as Prometheus metrics.

Deployment Agent (built, not yet production-validated)

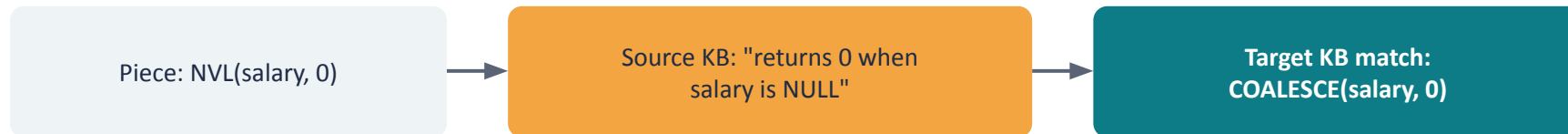


Implemented with production Terraform modules and Helm charts; awaiting live production validation.

7. RAG in the Schema Agent (CrackSQL)



Example



Both knowledge bases hold dialect function/keyword descriptions embedded in PGVector (Titan). Source: CrackSQL README (OpenDataBox)

Docker images and local infrastructure

Service (docker-compose)	Role	Endpoint
postgres-metadata (PostgreSQL)	Job metadata, checkpoints, PGVector KB	localhost:5432 / migration_metadata
postgres-sample (PostgreSQL)	Sample source/target	localhost:5433 / sample_source
mysql-sample (MySQL)	Sample source/target	localhost:3306 / sample_source
oracle-sample (Oracle XE)	Sample source/target (first boot 5-10 min)	localhost:1521 / XE
seatunnel (custom build)	Bulk-load engine with JDBC driver jars	infra/docker/seatunnel/
pgAdmin	Browse the PostgreSQL instances	localhost:5050

Exact image tags live in docker-compose.yml. Run: `docker-compose up -d`; `docker compose build seatunnel`. Production uses Terraform (VPC/EKS/RDS/IAM) and Helm.

Technology stack



Docker



PostgreSQL



MySQL

Oracle



Python 3.12



FastAPI



Streamlit



LangChain / LangGraph

AWS



Terraform



Kubernetes



GitHub Actions



Prometheus



Grafana



Poetry

Bedrock Nova Pro

Apache SeaTunnel

CrackSQL